

MAC 233: Arduino Introduction

Dr Max Champneys

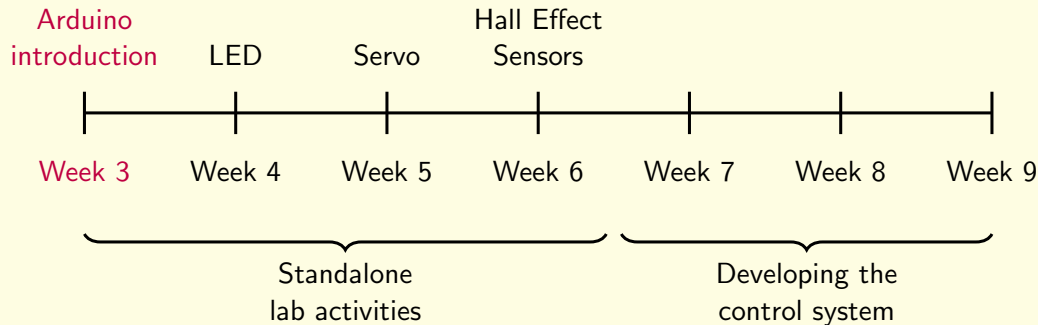
School of Mechanical, Aerospace, and Civil Engineering,
University of Sheffield, UK

MAC 233: **14/10/2026**



University of
Sheffield

MAC 233: LABS OVERVIEW



WHAT DO WE MEAN BY 'ARDUINO'?

The word 'Arduino' gets used for three different things:

1. **The platform** — An open-source project and company producing hardware designs, software and documentation.
2. **The board** — The physical piece of electronics in front of you. We will use the 'Arduino Nano Every' board in this course.
3. **The language** — The C++ based language, and its libraries, used to write your programs (called sketches).

Arduino

The platform

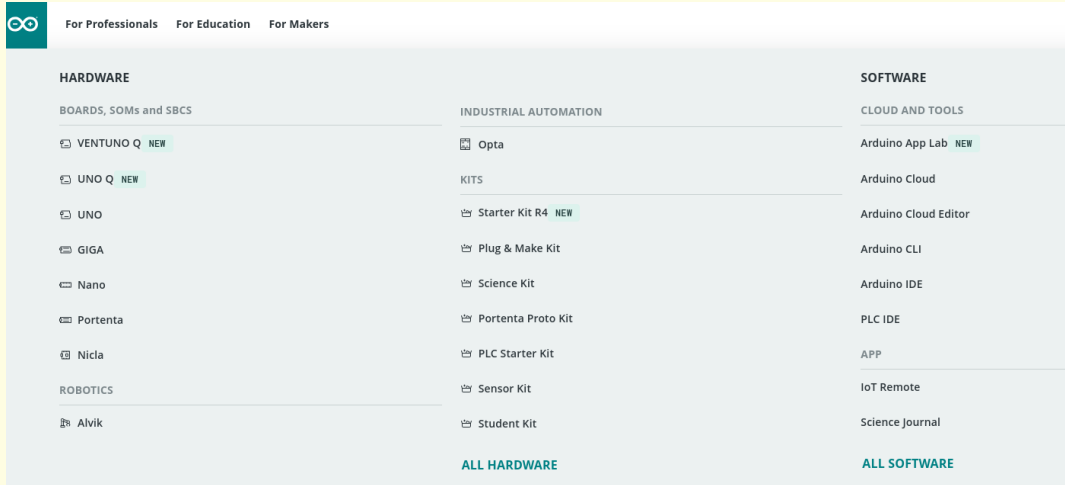


University of
Sheffield

Arduino designs, manufactures, and supports electronic devices and software, allowing people around the world to easily access advanced technologies that interact with the physical world. Our products are straightforward, simple, and powerful, ready to satisfy users' needs from students to makers and all the way to professional developers.

Adapted from: <https://www.arduino.cc/en/about/>

AN ECOSYSTEM OF HARDWARE AND SOFTWARE



The screenshot displays the Arduino website's main navigation and product categories. At the top, the Arduino logo is on the left, and three navigation links are centered: "For Professionals", "For Education", and "For Makers". Below this, the page is divided into three main sections: "HARDWARE", "SOFTWARE", and "CLOUD AND TOOLS".

HARDWARE

- BOARDS, SOMs and SBCs**
 - VENTUNO Q **NEW**
 - UNO Q **NEW**
 - UNO
 - GIGA
 - Nano
 - Portenta
 - Nicla
- ROBOTICS**
 - Alvik

INDUSTRIAL AUTOMATION

- Opta
- KITS**
 - Starter Kit R4 **NEW**
 - Plug & Make Kit
 - Science Kit
 - Portenta Proto Kit
 - PLC Starter Kit
 - Sensor Kit
 - Student Kit

ALL HARDWARE

SOFTWARE

- CLOUD AND TOOLS**
 - Arduino App Lab **NEW**
 - Arduino Cloud
 - Arduino Cloud Editor
 - Arduino CLI
 - Arduino IDE
 - PLC IDE
- APP**
 - IoT Remote
 - Science Journal

ALL SOFTWARE

<https://www.arduino.cc/>

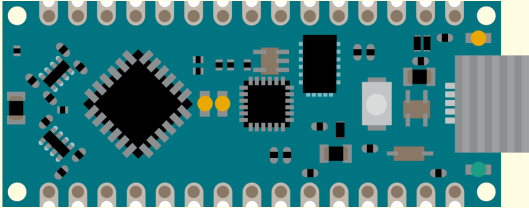
Arduino

The board



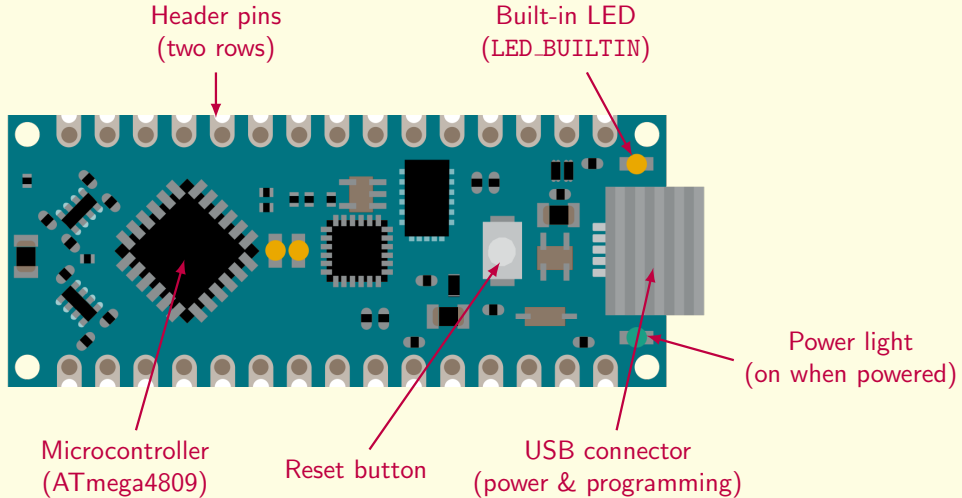
University of
Sheffield

ARDUINO NANO EVERY



Source: <https://docs.arduino.cc/hardware/nano-every/>

ANATOMY OF THE BOARD

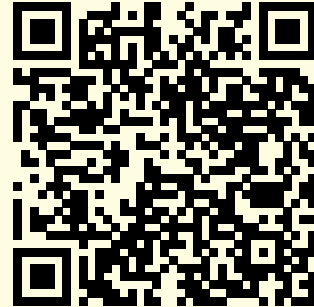
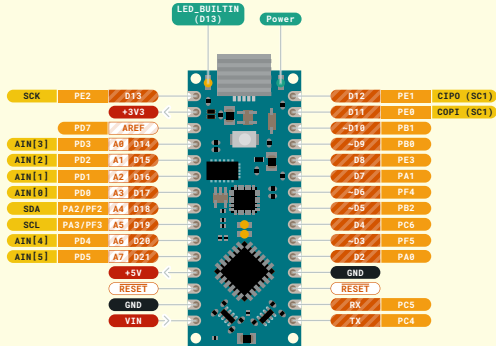


Inputs and outputs from the Arduino's microcontroller are handled by the 'pin's on the board.

Each pin is numbered and can be configured as an input or an output.

Because each pin connects to a different part of the microcontroller, different pins support different features.

PINOUT DIAGRAM



Source: <https://docs.arduino.cc/resources/pinouts/ABX00028-full-pinout.pdf>

Arduino

The language



University of
Sheffield

Arduino uses a C++ based programming language.

Arduino programs are called 'sketches'.

The Arduino language has some similarities and some important differences from the MATLAB you have seen before.

MATLAB is a dynamically typed language.

This means you can simply declare a variable and MATLAB will figure out what kind of memory is needed automatically:

```
x = 10;
```

Arduino is a statically typed language.

This means when you declare a variable, you must let Arduino know what type of data is contained within the variable:

```
int x = 10;
```

THE DATA TYPES YOU WILL NEED

```
int    count      = 10;           // -32,768 to 32,767
long   big_count  = 3000000L;    // +/- 2.1 billion
float  distance    = 5.6;        // decimal number
bool   ready      = false;       // true or false
char   letter      = 'M';        // one character
char   name[3]     = {'M', 'A', 'X'};

unsigned int rotations = 40000;   // no sign, twice the range
const int LED_PIN = 5;           // read-only, cannot change
```

CONTROL FLOW: 'IF', 'ELSE IF', 'ELSE'

Code inside the curly brackets runs only when the condition is true. Compare values with `==`, `!=`, `<`, `<=`, `>` or `>=`, and chain conditions with 'and' (`&&`) or 'or' (`||`).

```
int x = 10;
if (x > 21) {
    Serial.println("X is bigger than 21");
} else if (x >= 11 && x <= 20) {
    Serial.println("X is between 11 and 20");
} else if (x != 10) {
    Serial.println("X is less than 9");
} else {
    Serial.println("X is 10");
}
```

PINS: SETTING THE MODE

Before a pin can be used, the Arduino has to be told which direction it works in. 'pinMode' sets a pin to 'INPUT' or 'OUTPUT', and belongs in 'setup' — it only needs doing once.

```
void setup() {  
    pinMode(3, INPUT);  
    pinMode(5, OUTPUT);  
}
```

PINS: READING AND WRITING VALUES

'digitalWrite' sets an output pin 'HIGH' (on) or 'LOW' (off). 'digitalRead' gives back the 'HIGH' or 'LOW' state of an input pin. These usually belong in 'loop', where they happen over and over.

```
void loop() {  
    // turn 'on' the LED attached to pin 5  
    digitalWrite(5, HIGH);  
    // turn 'off' the LED attached to pin 5  
    digitalWrite(5, LOW);  
  
    // read the switch attached to pin 3  
    int switch_state = digitalRead(3);  
}
```

Sketches

Arduino programs



University of
Sheffield

SKETCH BASICS

Arduino has two special functions that must exist in a sketch before it can be compiled and uploaded to your board: 'setup' and 'loop'.

'File' → 'Examples' → '01.Basics' → 'BareMinimum':

```
void setup() {  
    // put your setup code here, to run once:  
}
```

```
void loop() {  
    // put your main code here, to run repeatedly:  
}
```

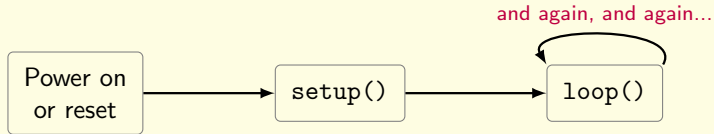
SKETCH BASICS

```
void setup() {  
  // ...  
}
```

← runs **once**, the moment the board is powered on or reset

```
void loop() {  
  // ...  
}
```

← runs **forever** afterwards — as soon as it finishes, it starts again



WHAT GOES IN EACH FUNCTION?

setup() → 'get ready'

Anything that only needs doing once, before the board starts doing its job:

- ▶ choose whether a pin is an input or an output ('pinMode')
- ▶ open a serial connection to the computer ('Serial.begin')
- ▶ attach a servo, or start up a sensor library
- ▶ give global variables their starting values

loop() → 'do the work'

Anything that has to happen over and over for as long as the board is powered:

- ▶ read the inputs ('digitalRead', 'analogRead')
- ▶ decide what to do with 'if' / 'else if' / 'else'
- ▶ drive the outputs ('digitalWrite', 'analogWrite')
- ▶ wait before the next pass ('delay')

VARIABLES: SCOPE

The **scope** of a variable is the part of the sketch in which it can be accessed: declared outside any function it belongs to the whole program, declared inside a function it belongs to that function alone.

```
// global: usable anywhere
```

```
int wheel_rotations = 0;
```

← Global scope — visible
anywhere in the sketch

```
void setup() {
```

```
    // local: setup only
```

```
    int setup_rotations = 1;
```

```
}
```

← Local scope — visible only
inside this function

```
void loop() {
```

```
    wheel_rotations = 3;    // ok
```

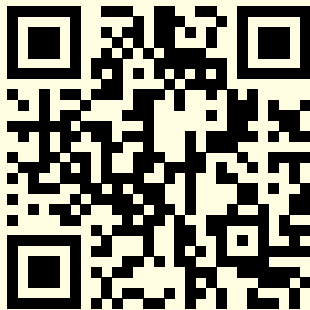
```
    setup_rotations = 4;    // error
```

```
}
```

← *setup_rotations* does not exist
out here — the sketch will not compile

ADDITIONAL INFORMATION

We have only covered the most useful variables and structure for our use case; however, there are many more options available:



<https://docs.arduino.cc/language-reference>

Lab 1

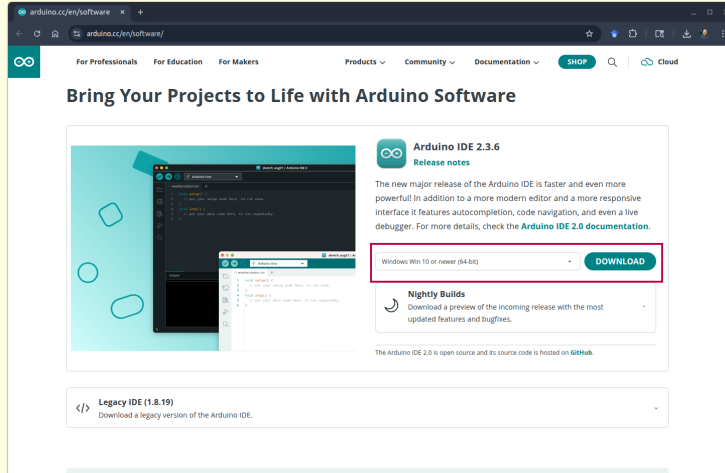
Communicating with the internal LED



University of
Sheffield

STEP 1: DOWNLOAD AND INSTALL ARDUINO IDE

Go to `https://www.arduino.cc/en/software`



STEP 2: GET THE LAB 1 WORKSHEET

Everything you need for the labs lives in the course GitHub repository. Scan the code, or follow the link, to open the Lab 1 worksheet:

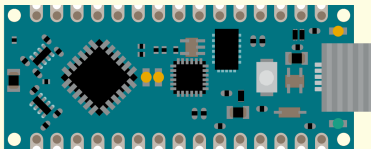


<https://github.com/MDCHAMP/MAC233/blob/main/labs/lab-1-blink/docs/lab-1-blink.pdf>

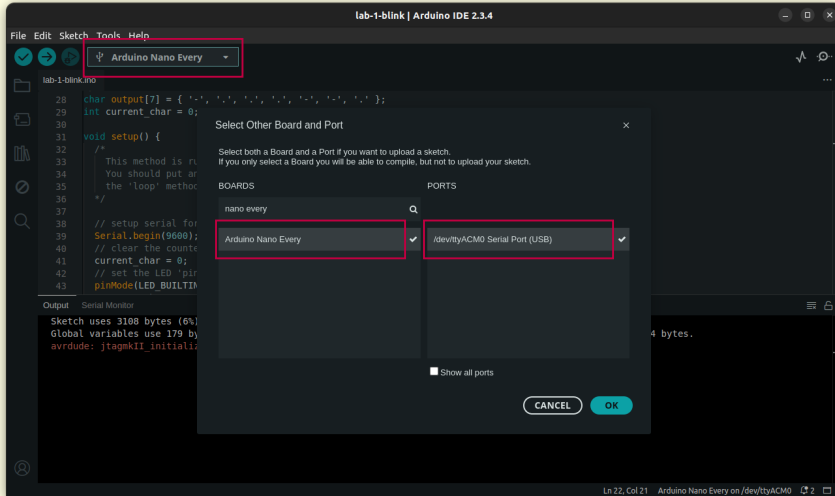
STEP 3: CONNECT YOUR ARDUINO

Plug your Arduino into your computer using a USB cable.

A light on the board should come on to show that it has power. If nothing lights up, try a different USB lead — some leads only carry power for charging and cannot carry data.



STEP 4: SELECT THE BOARD AND PORT



STEP 5: WRITE, COMPILE AND UPLOAD YOUR SKETCH

Compile: 'Sketch' → 'Verify\Compile'.

This checks your code for errors without sending anything to the board.

Upload: 'Sketch' → 'Upload'.

This compiles the sketch and then sends it to the board.

A red warning about being unable to locate 'flash' is normal on the Nano Every and can be ignored.

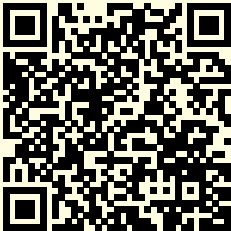
STEP 6: OPEN THE SERIAL MONITOR

The Serial Monitor shows you the messages your Arduino sends back: 'Tools' → 'Serial Monitor'.

It opens in the bottom section of the IDE, next to the 'Output' tab.

Set the baud rate drop-down to **9600**. This must match the value you passed to 'Serial.begin' in your sketch, not, you will see random characters instead of words!

Lab 1 worksheet



Nano Every pinout



Problems? Raise your hand and someone will come and help you.